

Vision-Language Model Fine-Tuning

A Complete Practical Guide

From Zero to Training Your Own Robotics VLM

Fine-tuning InternVL3-8B on RoboVQA with Curriculum Learning

What You'll Learn:

- How Vision-Language Models process multimodal data
- QLoRA: Training 8B+ parameter models on consumer hardware
- Curriculum learning for robotics scene understanding
- Complete pipeline from raw data to trained model
- Real-world results and lessons learned

January 2026

Contents

1	Introduction: What Are We Building?	4
1.1	The Goal	4
1.2	Why This Matters	4
1.3	VLMs vs VLAs: An Important Distinction	4
1.4	Project Overview	4
2	Understanding Vision-Language Models	5
2.1	The Architecture (Simplified)	5
2.2	How Images Become Tokens	5
2.3	The Conversation Format	5
3	The Hardware Reality Check	6
3.1	Memory Requirements	6
3.2	Our Setup: DGX Spark	6
3.3	Flash Attention on New Hardware: A Cautionary Tale	7
4	Choosing Your Model	7
4.1	Model Evaluation	7
4.2	Why InternVL3-8B?	7
4.3	Model Size Trade-offs	8
5	Understanding Your Dataset: RoboVQA	8
5.1	Dataset Overview	8
5.2	Task Type Distribution	8
5.3	The QA Text Format	8
6	Data Preprocessing: The Make-or-Break Step	9
6.1	Why Preprocessing Matters	9
6.2	Our Preprocessing Pipeline	9
6.3	Step 1: Frame Extraction	9
6.4	Step 2: JSONL Format	9
6.5	Stage 1 vs Stage 2 Format	10
6.6	Final Dataset Statistics	10
7	QLoRA: Training Large Models Efficiently	10
7.1	The Problem	10
7.2	The Solution: QLoRA	11
7.2.1	4-bit Quantization	11
7.2.2	Low-Rank Adaptation (LoRA)	11
7.2.3	Paged Optimizers	11
7.3	LoRA Configuration	11
7.4	Why These Target Modules?	11
7.5	Memory Savings Summary	12
8	Curriculum Learning: Why Order Matters	12
8.1	The Core Insight	12
8.2	Our Two-Stage Curriculum	12
8.2.1	Stage 1: Visual Grounding (Foundation)	12
8.2.2	Stage 2: Multi-Turn Reasoning (Advanced)	12
8.3	Why Not Skip Stage 1?	13

8.4	Stage Transition Strategy	13
9	The Training Pipeline	13
9.1	Project Structure	13
9.2	The DataLoader	14
9.2.1	Dataset Class	14
9.2.2	Collator	14
9.3	The Training Loop	14
10	Hyperparameters Explained	15
10.1	Complete Configuration	15
10.2	Key Hyperparameter Explanations	15
10.2.1	Batch Size and Gradient Accumulation	15
10.2.2	Learning Rate	15
10.2.3	Learning Rate Schedule	16
10.2.4	Sequence Length Calculation	16
11	Engineering for Long-Running Training Jobs	16
11.1	The Mindset Shift	16
11.2	Checkpoint Strategy: The Most Important Decision	16
11.2.1	Our Costly Mistake	16
11.2.2	The Corrected Strategy	17
11.2.3	Checkpoint Frequency Guidelines	17
11.2.4	Checkpoint Retention	17
11.2.5	Always Save on Completion	18
12	Monitoring and Observability	18
12.1	Integrated Profiling: Not Optional	18
12.1.1	Per-Step Timing Metrics	18
12.1.2	What Each Metric Tells You	19
12.1.3	CSV Logging	19
12.2	Key Health Indicators	20
12.3	Verify Checkpoint Loading	20
12.4	Automated Health Monitoring	21
13	Debugging Crashes on New Hardware	21
13.1	Our Crash Pattern	21
13.2	System Diagnostic Tools	22
13.2.1	dmesg: The First Check (But Often Useless)	22
13.2.2	journalctl: Find Previous Boot Logs	22
13.2.3	Kernel Logs: The Smoking Gun	22
13.2.4	Thermal Monitoring	22
13.3	The Root Cause: Unified Memory and Variable-Length Sequences	23
13.3.1	Why Stage 2 Crashed But Stage 1 Didn't	23
13.3.2	The Eval Batch Size Trap	23
13.3.3	Memory Headroom	23
13.4	Debugging Tensor Mismatches and NaN Losses	23
14	Evaluation and Validation	24
14.1	10% Subset Validation	24
14.2	Full-Scale Results	24
14.2.1	Stage 1: Visual Grounding (100% Data)	24

15 Battle-Tested Pitfalls and How to Avoid Them	25
15.1 Data Preprocessing Pitfalls	25
15.2 Checkpoint and Recovery Pitfalls	25
15.3 Memory and Stability Pitfalls	26
15.4 Stage Transition Pitfalls	26
16 Results and Lessons Learned	26
16.1 Final Results Summary	26
16.2 Key Lessons Learned	27
17 Appendix: Running the Code	28
17.1 Installation	28
17.2 Preprocessing	28
17.3 Training	28
17.4 Configuration Override	29
18 Conclusion	29

1 Introduction: What Are We Building?

1.1 The Goal

We are fine-tuning **InternVL3-8B**, an 8-billion parameter Vision-Language Model, on the **RoboVQA dataset** to create a model that can understand robotics scenes and answer questions about them. Think of it as teaching an AI to look at robot camera footage and intelligently answer questions like:

- “What objects can the robot grasp?” (Affordance detection)
- “What should the robot do next to complete this task?” (Planning)
- “Did the robot successfully pick up the object?” (Success prediction)

1.2 Why This Matters

Vision-Language Models (VLMs) represent the cutting edge of multimodal AI. They can process both images and text, understanding the relationship between what they see and what they’re asked. For robotics, this capability is transformative—instead of hand-coding every visual pattern, we can train models to understand visual scenes naturally.

1.3 VLMs vs VLAs: An Important Distinction

Key Insight

Before diving in, let’s clarify a common confusion:

- **VLM (Vision-Language Model)**: Understands images and generates text. Great for reasoning, planning, and understanding scenes.
- **VLA (Vision-Language-Action Model)**: Outputs actual robot control signals (joint positions, velocities, etc.).

Our fine-tuned VLM will NOT directly control a robot. Instead, it serves as a **reasoning module** in a larger system—understanding scenes and providing high-level guidance that other components translate into actions.

1.4 Project Overview

Component	Choice
Base Model	InternVL3-8B
Dataset	RoboVQA (221,912 video samples)
Training Method	QLoRA (4-bit quantization + LoRA)
Training Strategy	Two-stage curriculum learning
Hardware	DGX Spark (GB10 Blackwell, 128GB)
Training Time	~25 days (Stage 1) + ~22 days (Stage 2, including crash recovery)

Table 1: Project configuration summary

2 Understanding Vision-Language Models

2.1 The Architecture (Simplified)

A VLM has three main components working together:

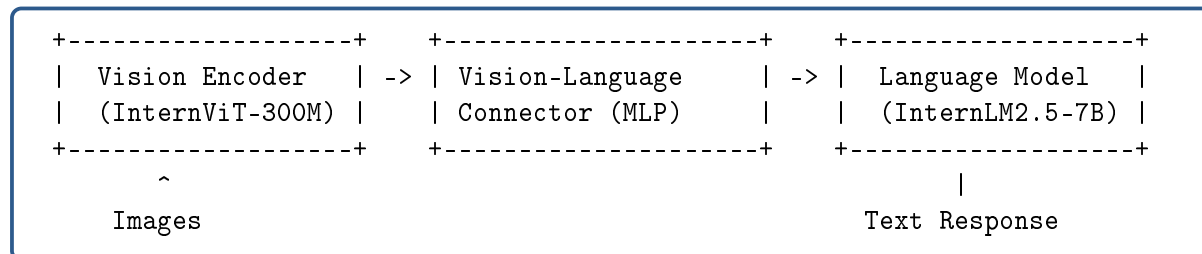


Figure 1: VLM architecture overview

Vision Encoder: Takes images and converts them into “visual tokens”—numerical representations that capture what’s in the image. InternVL3-8B uses InternViT-300M, which processes images at 448×448 resolution.

Vision-Language Connector: A bridge (typically an MLP) that translates visual tokens into a format the language model understands. Think of it as a translator between “image speak” and “text speak.”

Language Model: The brain of the operation. It takes the translated visual information plus your text prompt and generates a response. InternVL3-8B uses InternLM2.5-7B as its language backbone.

2.2 How Images Become Tokens

When you feed an image to InternVL3, the following transformation occurs:

1. The image is resized to 448×448 pixels
2. It’s divided into patches (like cutting a photo into a grid)
3. Each patch becomes a token (typically 256 tokens per image)
4. Multiple images = multiple sets of tokens

For video data (like our robotics data with 16 frames):

$$\text{Visual Tokens} = \text{Frames} \times \text{Tokens per Frame} = 4 \times 256 = 1,024 \text{ tokens} \quad (1)$$

Warning

This is critical for memory calculations—more frames = more tokens = more GPU memory. We use 4 frames in Stage 1 and 10 frames in Stage 2 to balance temporal context with memory constraints.

2.3 The Conversation Format

VLMs are trained on conversations. Here’s what a training sample looks like:

```

1 {
2   "conversations": [
3     {
4       "from": "human",
5       "value": "<image><image><image><image>Looking at these frames
6               from a robot camera, what objects can the robot reach?"
7     },
8     {
9       "from": "gpt",
10      "value": "The robot can currently reach a red ball on the table
11              and a blue cup positioned to its right."
12     }
13   ]
14 }

```

Listing 1: Conversation format for VLM training

The `<image>` tags are placeholders that get replaced with actual visual tokens during training. The model learns to associate visual patterns with the corresponding text responses.

3 The Hardware Reality Check

3.1 Memory Requirements

Training VLMs is memory-intensive. Here’s a breakdown for InternVL3-8B:

Component	QLoRA	Full Fine-tune
Model weights	~5 GB (4-bit)	~16 GB (bfloat16)
LoRA adapters	~0.2 GB	N/A
Optimizer states	~0.3 GB	~32 GB
Gradients	~0.2 GB	~16 GB
Activations (4 frames)	~2–4 GB	~8–16 GB
Total	~10 GB	~72+ GB

Table 2: Memory requirements comparison

This is why we use QLoRA—it makes training possible on reasonable hardware.

3.2 Our Setup: DGX Spark

We trained on NVIDIA DGX Spark with:

- **GPU:** GB10 Blackwell (128GB unified memory)
- **CUDA:** 13.0
- **Architecture:** sm_121

The unified memory architecture shares memory between CPU and GPU, which changes some optimization strategies.

3.3 Flash Attention on New Hardware: A Cautionary Tale

Key Insight

Cutting-edge hardware doesn't always work with cutting-edge software. Always benchmark!

We attempted to compile Flash Attention 2 for the GB10 (sm_121), which required:

1. Adding sm_121 to the build configuration
2. Updating CUTLASS to v4.3.0 for CUDA 13.0 compatibility
3. Patching CUDA type definitions
4. Modifying runtime architecture checks

Result: It compiled and ran, but benchmarking revealed surprising results:

Implementation	Latency (512 tokens)
Flash Attention 2	489ms
Flash Attention 3	489ms
PyTorch SDPA	481ms

Table 3: Attention implementation benchmarks on GB10

SDPA was actually 2% faster! The Flash Attention kernels weren't optimized for Blackwell's architecture. **Lesson:** Always benchmark; don't assume newer = better.

4 Choosing Your Model

4.1 Model Evaluation

We evaluated three candidates for video-based robotics VQA:

Model	Video Support	DGX Spark Support	Decision
PaliGemma 3B	Requires frame extraction	No official support	Rejected
Qwen2.5-VL-7B	Native video	Community only	Considered
InternVL3-8B	Native video	Official playbooks	Selected

Table 4: Model comparison for our use case

4.2 Why InternVL3-8B?

Key factors that drove our decision:

1. NVIDIA provides official fine-tuning playbooks for InternVL3 on their hardware
2. Native multi-image input matches RoboVQA's 16-frame format
3. Strong embodied AI benchmarks
4. Well-documented QLoRA recipes
5. 8B parameters fit comfortably with QLoRA in 128GB unified memory

4.3 Model Size Trade-offs

Bigger isn't always better for robotics tasks:

- **InternVL3-1B**: Fast inference, limited reasoning
- **InternVL3-2B**: Good balance for simple tasks
- **InternVL3-8B**: Strong reasoning, slower inference (our choice)
- **Larger models**: Diminishing returns for robotics tasks

For robotics VQA, we need enough capacity to understand complex spatial relationships but don't need the reasoning depth of a 70B model.

5 Understanding Your Dataset: RoboVQA

5.1 Dataset Overview

RoboVQA contains **221,912 video samples** of robot manipulation tasks. Each sample includes:

- 16 frames of video at various resolutions
- Multiple QA pairs covering different aspects of the scene
- Task annotations categorizing each question type

5.2 Task Type Distribution

Task Type	Description	Percentage
Affordance Detection	“What can the robot interact with?”	33%
Planning Variants	“What should happen next?”	35%
Success Prediction	“Did the task succeed?”	14%
Future Prediction	“What will the scene look like?”	11%
Past Description	“What just happened?”	6%

Table 5: RoboVQA task type distribution

5.3 The QA Text Format

RoboVQA uses a structured format for its question-answer pairs:

```

1 <task:affordance:discriminative:discrete>
2 Looking at the scene, can the robot reach the red cup?
3 Q: Can the robot reach the red cup? A: Yes

```

Listing 2: RoboVQA text format

The `<task:type:subtype:format>` marker tells us:

- **type**: Main category (affordance, planning, etc.)
- **subtype**: Discriminative (yes/no) vs. generative (open-ended)
- **format**: Discrete answers vs. freeform text

This structure is important for preprocessing—we need to parse it correctly.

6 Data Preprocessing: The Make-or-Break Step

6.1 Why Preprocessing Matters

Raw data rarely matches what models expect. Poor preprocessing leads to:

- Training failures (wrong format)
- Poor performance (information loss)
- Wasted compute (inefficient storage)

6.2 Our Preprocessing Pipeline

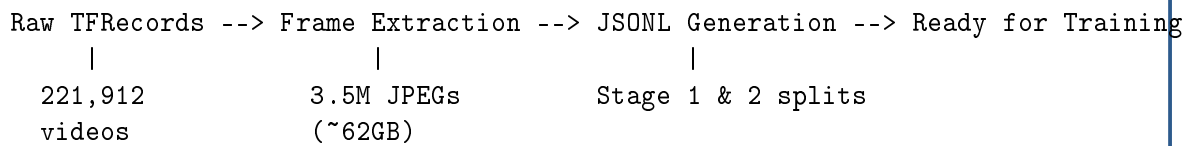


Figure 2: Preprocessing pipeline overview

6.3 Step 1: Frame Extraction

```

1 for sample in tfrecord:
2     for i, frame in enumerate(sample.frames):
3         # Resize to model's expected size
4         img = resize(frame, (448, 448))
5         # Save with quality balance
6         img.save(f"{unique_id}_{i:02d}.jpg", quality=85)
  
```

Listing 3: Frame extraction pseudocode

Tip

Key decisions for frame extraction:

- **448×448 resolution:** Matches InternVL3 input size
- **JPEG quality 85:** Good balance of size vs. quality
- **Shared image folder:** Both stages reference same images (saves ~60GB)

6.4 Step 2: JSONL Format

The training format follows InternVL conventions:

```

1 {
2     "id": "video_001_qa_0",
3     "images": [
4         "images/video_001_00.jpg",
5         "images/video_001_01.jpg",
6         "images/video_001_02.jpg",
7         "images/video_001_03.jpg"
8     ],
9     "conversations": [
  
```

```

10     {"from": "human", "value": "<image><image><image><image>What can the robot
11     grasp?"},
12     {"from": "gpt", "value": "The robot can grasp the red ball on the table."}
13 ],
14 "metadata": {"task_type": "affordance"}

```

Listing 4: JSONL training format

6.5 Stage 1 vs Stage 2 Format

Stage 1 (Visual Grounding): One QA pair per sample

```

1 {
2   "conversations": [
3     {"from": "human", "value": "<image>...<image> Question 1?"},
4     {"from": "gpt", "value": "Answer 1"}
5   ]
6 }

```

Listing 5: Stage 1 format: Single QA

Stage 2 (Multi-Turn): All QAs from one video as conversation

```

1 {
2   "conversations": [
3     {"from": "human", "value": "<image>...<image> Question 1?"},
4     {"from": "gpt", "value": "Answer 1"},
5     {"from": "human", "value": "Question 2?"},
6     {"from": "gpt", "value": "Answer 2"},
7     {"from": "human", "value": "Question 3?"},
8     {"from": "gpt", "value": "Answer 3"}
9   ]
10 }

```

Listing 6: Stage 2 format: Multi-turn conversation

6.6 Final Dataset Statistics

Stage	Split	Samples
Stage 1	Train	722,979
Stage 1	Validation	80,330
Stage 2	Train	199,721
Stage 2	Validation	22,191
Images		3.5M JPEGs (~62GB)

Table 6: Final preprocessed dataset statistics

7 QLoRA: Training Large Models Efficiently

7.1 The Problem

Full fine-tuning of an 8B model requires:

- ~16GB for model weights (bfloat16)
- ~32GB for optimizer states (Adam stores 2 copies)

- $\sim 16\text{GB}$ for gradients
- Variable activation memory

Total: 64GB+ just for basics, before adding batch data.

7.2 The Solution: QLoRA

QLoRA combines three powerful techniques:

7.2.1 4-bit Quantization

Instead of 16 bits per parameter, we use 4 bits:

$$\text{Memory} = 8\text{B params} \times 4 \text{ bits} = 4\text{GB} \quad (\text{vs. } 16\text{GB for bfloat16}) \quad (2)$$

7.2.2 Low-Rank Adaptation (LoRA)

Instead of updating all parameters, we add small “adapter” matrices:

$$\begin{aligned} \text{Original : } W &\in \mathbb{R}^{4096 \times 4096} = 16\text{M parameters} \\ \text{LoRA : } A &\in \mathbb{R}^{4096 \times 128} + B \in \mathbb{R}^{128 \times 4096} = 1\text{M parameters} \end{aligned} \quad (3)$$

This is a **94% reduction** in trainable parameters!

7.2.3 Paged Optimizers

Optimizer states are moved to CPU RAM when not needed, reducing GPU memory pressure.

7.3 LoRA Configuration

```

1 lora_r: 128           # Rank of adaptation matrices
2 lora_alpha: 256       # Scaling factor (typically 2x rank)
3 lora_dropout: 0.05    # Regularization
4 target_modules:       # Which layers to adapt
5   - q_proj           # Query projection
6   - k_proj           # Key projection
7   - v_proj           # Value projection
8   - o_proj           # Output projection
9   - gate_proj         # MLP gate
10  - up_proj           # MLP up projection
11  - down_proj          # MLP down projection

```

Listing 7: LoRA configuration

7.4 Why These Target Modules?

We adapt the **language model** attention and MLP layers because:

1. They contain most of the learned knowledge
2. Vision encoder benefits less from task-specific adaptation
3. Connector is small enough to train directly if needed

7.5 Memory Savings Summary

Configuration	GPU Memory
Full fine-tune	72+ GB
LoRA only	~24 GB
QLoRA	~10–12 GB

Table 7: Memory comparison across training methods

QLoRA makes training possible on consumer GPUs (RTX 3090, RTX 4090) or workstations.

8 Curriculum Learning: Why Order Matters

8.1 The Core Insight

Key Insight

Imagine teaching someone to write essays:

- **Bad approach:** Give them complex essay prompts immediately
- **Good approach:** First teach sentences, then paragraphs, then essays

The same principle applies to VLM training.

8.2 Our Two-Stage Curriculum

8.2.1 Stage 1: Visual Grounding (Foundation)

- One question-answer pair per video
- Teaches: Object recognition, spatial understanding, basic scene comprehension
- 722,979 training samples
- **Goal:** Model learns to “see” correctly

8.2.2 Stage 2: Multi-Turn Reasoning (Advanced)

- Multiple QA pairs per video as conversation
- Teaches: Reasoning chains, task relationships, context maintenance
- 199,721 training samples
- Initialized from Stage 1 checkpoint
- **Goal:** Model learns to “think” about what it sees

8.3 Why Not Skip Stage 1?

We ran an ablation experiment to find out:

Training Approach	Stage 1 Eval Loss	Stage 2 Eval Loss	Difference
Curriculum (S1→S2)	0.334 / 1.40 ppl	0.034 / 1.03 ppl	—
Stage 2 Only	0.495 / 1.64 ppl	0.043 / 1.04 ppl	—
Degradation	+48% loss	+26% loss	

Table 8: Curriculum learning ablation results

Warning

Without Stage 1: The model struggles with basic visual understanding, which cascades into worse reasoning performance.

Conclusion: The visual grounding stage is essential. You cannot build reasoning on a shaky visual foundation.

8.4 Stage Transition Strategy

When moving from Stage 1 to Stage 2:

1. **Load Stage 1 weights:** Start where we left off
2. **Reset learning rate:** Fresh cosine schedule (but lower initial LR)
3. **Reset best loss tracking:** Stage 2 metrics are different
4. **Keep step counter:** For logging continuity

```

1 # Stage 2 configuration
2 resume_from_checkpoint: experiments/stage1/checkpoints/final
3 reset_scheduler_on_resume: true # Fresh LR schedule
4 learning_rate: 2.0e-5 # Lower than Stage 1's 4e-5

```

Listing 8: Stage 2 configuration

9 The Training Pipeline

9.1 Project Structure

```

1 vlm-ft/
2 |-- vlmft/
3 |   |-- data/
4 |   |   |-- dataloader.py           # Dataset and collation
5 |   |   +-- preprocess_robovqa.py   # Raw data processing
6 |   |-- models/
7 |   |   +-- internvl.py             # Model loading and QLoRA setup
8 |   |-- training/
9 |   |   |-- trainer.py              # Training loop
10 |   |   +-- config.py              # Configuration dataclasses
11 |   +-- scripts/
12 |       |-- train.py               # Training entry point
13 |       +-- eval.py               # Evaluation entry point
14 |-- configs/

```

```

15 | |-- stage1.yaml                # Stage 1 hyperparameters
16 | |-- stage2.yaml                # Stage 2 hyperparameters
17 | +-- stage2_only.yaml          # Ablation: skip Stage 1
18 +-- experiments/                # Output directory

```

Listing 9: Repository structure

9.2 The DataLoader

9.2.1 Dataset Class

```

1 class RoboVQADataset(Dataset):
2     def __getitem__(self, idx):
3         sample = self.samples[idx]
4         images = [Image.open(path) for path in sample["images"]]
5         return {
6             "images": images,
7             "question": sample["conversations"][0]["value"],
8             "answer": sample["conversations"][1]["value"],
9         }

```

Listing 10: Dataset implementation

9.2.2 Collator

```

1 class InternVLCollator:
2     def __call__(self, batch):
3         # Process images through model's vision preprocessing
4         pixel_values = self.preprocess_images(batch)
5
6         # Tokenize conversations
7         input_ids, labels = self.tokenize_conversations(batch)
8
9         # Create attention mask and image flags
10        return {
11            "pixel_values": pixel_values,
12            "input_ids": input_ids,
13            "attention_mask": attention_mask,
14            "labels": labels,
15            "image_flags": image_flags,
16        }

```

Listing 11: Collator implementation

9.3 The Training Loop

```

1 for epoch in range(num_epochs):
2     for batch in train_dataloader:
3         # Forward pass with mixed precision
4         with torch.amp.autocast("cuda", dtype=torch.bfloat16):
5             outputs = model(**batch)
6             loss = outputs.loss / gradient_accumulation_steps
7
8         # Backward pass
9         loss.backward()
10
11        # Update weights every N steps (gradient accumulation)
12        if step % gradient_accumulation_steps == 0:
13            torch.nn.utils.clip_grad_norm_(params, max_grad_norm)

```

```

14         optimizer.step()
15         scheduler.step()
16         optimizer.zero_grad()
17
18     # Periodic evaluation
19     if step % eval_steps == 0:
20         eval_metrics = evaluate(model, eval_dataloader)
21         save_if_best(model, eval_metrics)

```

Listing 12: Training loop pseudocode

10 Hyperparameters Explained

10.1 Complete Configuration

Parameter	Stage 1	Stage 2	Notes
LoRA rank (r)	128	128	InternVL official
LoRA alpha	256	256	$2 \times r$ heuristic
LoRA dropout	0.05	0.05	—
Learning rate	4e-5	2e-5	Lower for continued training
Batch size	2	2	Memory-optimized
Gradient accumulation	16	16	Effective batch: 32
Epochs	1	1	Prevent overfitting
Warmup ratio	0.03	0.03	—
LR scheduler	Cosine	Cosine	—
Max grad norm	1.0	1.0	—
Optimizer	AdamW	AdamW	$\beta_1=0.9$, $\beta_2=0.999$
Weight decay	0.01	0.01	—
Max frames	4	10	Memory vs. context
Max sequence length	2048	4096	Stage 2 needs longer
Attention	SDPA	SDPA	See Flash Attention section
Precision	bfloat16	bfloat16	Mixed precision

Table 9: Complete hyperparameter configuration

10.2 Key Hyperparameter Explanations

10.2.1 Batch Size and Gradient Accumulation

Problem: We want large batch sizes for stable training, but GPU memory limits us to `batch_size=2`.

Solution: Gradient accumulation simulates larger batches:

$$\text{Effective batch} = \text{batch_size} \times \text{gradient_accumulation_steps} = 2 \times 16 = 32 \quad (4)$$

The model sees 32 samples before each weight update, but only 2 are in memory at once.

10.2.2 Learning Rate

- **Stage 1:** Higher LR (4e-5) for initial learning
- **Stage 2:** Lower LR (2e-5) for refinement without catastrophic forgetting

10.2.3 Learning Rate Schedule

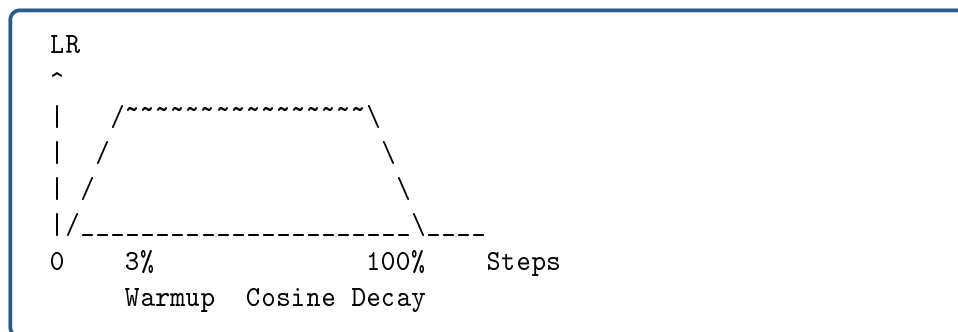


Figure 3: Learning rate schedule with warmup and cosine decay

10.2.4 Sequence Length Calculation

Memory calculation for Stage 1:

$$\text{Visual tokens} = 4 \text{ frames} \times 256 \text{ tokens/frame} = 1,024 \text{ tokens} \quad (5)$$

We set `max_length=2048` to allow 1,024 visual + 1,024 text tokens.

11 Engineering for Long-Running Training Jobs

Training VLMs at scale is not just a machine learning problem—it’s a *systems engineering* problem. Our Stage 1 ran for 25 days and Stage 2 consumed over 22 days of compute (including crash recovery). During that time, we experienced two system crashes, lost days of compute, and learned hard lessons about what it takes to run training jobs that span weeks. This section distills those lessons.

11.1 The Mindset Shift

Key Insight

When your training job runs for weeks, **everything that can go wrong will go wrong**. Power fluctuations, OS updates, memory spikes from edge-case samples, thermal events, WiFi drops causing NFS timeouts—you must design for failure from day one.

A 30-minute training job can afford sloppy engineering. A 30-day training job cannot. The discipline required is closer to running a production service than writing a research script.

11.2 Checkpoint Strategy: The Most Important Decision

11.2.1 Our Costly Mistake

We initially set `save_steps: 5000` for Stage 2. With 227 seconds per step, that meant:

$$5000 \times 227s = 1,135,000s \approx \mathbf{13.1 \text{ days between checkpoints}} \quad (6)$$

When the system crashed at step 27,186, our last checkpoint was at step 25,000. We lost **2,186 steps—roughly 6 days of compute**.

Warning

The checkpoint frequency calculation: Always convert `save_steps` to wall-clock time. The formula is simple:

Time between saves = `save_steps` × seconds/step

If this number is greater than 24 hours, your checkpoint frequency is too low.

11.2.2 The Corrected Strategy

After this painful lesson, we changed to `save_steps: 500`:

$$500 \times 227s = 113,500s \approx \mathbf{31 \text{ hours between checkpoints}} \quad (7)$$

When the system crashed again at step 28,929, our checkpoint at 28,500 meant we only lost **429 steps (~27 hours)**—still painful, but 5x better.

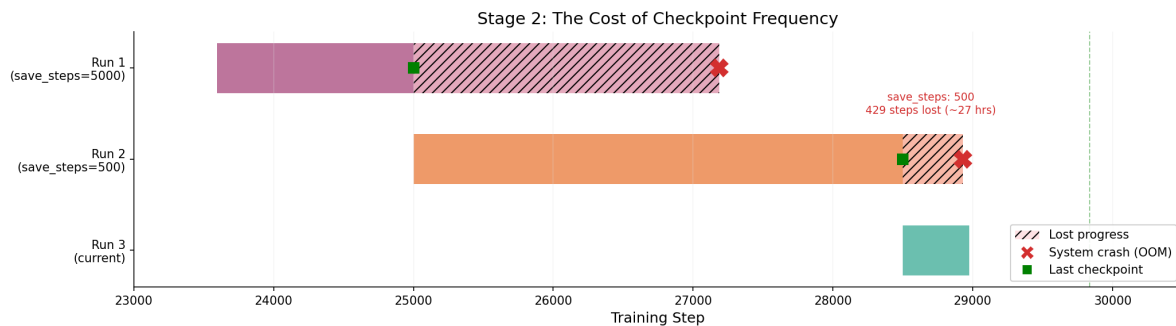


Figure 4: Stage 2 crash and recovery timeline. The hatched regions show lost progress—the difference between `save_steps: 5000` (6 days lost) and `save_steps: 500` (27 hours lost) is dramatic.

11.2.3 Checkpoint Frequency Guidelines

Step Time	Recommended <code>save_steps</code>	Time Between Saves
< 10s	2000–5000	6–14 hours
10–60s	1000–2000	3–33 hours
60–120s	500–1000	8–33 hours
> 120s	250–500	8–17 hours

Table 10: Checkpoint frequency recommendations by step time

Rule of thumb: You should never lose more than 24 hours of compute to a crash. Disk space is cheap; GPU time is not.

11.2.4 Checkpoint Retention

Use `save_total_limit` to avoid filling your disk. Keep the 3–5 most recent checkpoints plus the best checkpoint based on eval loss. Our directory structure:

```

1 checkpoints/
2   step_27500/    # Recent
3   step_28000/    # Recent

```

```

4  step_28500/      # Most recent
5  best/           # Symlink to best eval loss checkpoint
6  final/          # Saved at end of training

```

Listing 13: Checkpoint directory

11.2.5 Always Save on Completion

Our Stage 1 saved checkpoints every 1000 steps, but training ran for 2,259 steps. The final checkpoint was at step 2000—missing the last 259 steps. Always force a save at training completion, regardless of the step interval.

12 Monitoring and Observability

12.1 Integrated Profiling: Not Optional

We built timing instrumentation directly into the training loop, logging every metric to CSV files. This proved invaluable for diagnosing issues weeks into training.

12.1.1 Per-Step Timing Metrics

```

1  # Timing every phase of each training step
2  t_start = time.time()
3
4  # 1. Data loading
5  t0 = time.time()
6  batch = next(data_iter)
7  t_data = time.time() - t0
8
9  # 2. CPU -> GPU transfer
10 t0 = time.time()
11 batch = to_device(batch)
12 t_transfer = time.time() - t0
13
14 # 3. Forward pass
15 t0 = time.time()
16 loss = model(**batch).loss
17 t_forward = time.time() - t0
18
19 # 4. Backward pass
20 t0 = time.time()
21 loss.backward()
22 t_backward = time.time() - t0
23
24 # 5. Optimizer step (every N accumulation steps)
25 t0 = time.time()
26 optimizer.step()
27 t_optimizer = time.time() - t0
28
29 t_step = time.time() - t_start
30 throughput = batch_size / t_step
31 gpu_mem = torch.cuda.memory_allocated() / 1e9

```

Listing 14: Profiling instrumentation in the training loop

12.1.2 What Each Metric Tells You

Metric	Healthy Signal	Problem Indicators
<code>t_data</code>	< 0.1s (negligible)	> 1s: I/O bottleneck, slow disk, few workers
<code>t_transfer</code>	< 0.1s (negligible)	> 0.5s: Large batch or memory pressure
<code>t_forward</code>	Consistent	Spikes: variable-length sequences hitting max
<code>t_backward</code>	$\sim 1.5\text{--}2\times$ forward	$> 3\times$ forward: gradient checkpointing issue
<code>t_optimizer</code>	< 0.5s	Spikes: paged optimizer swapping to CPU
<code>gpu_mem_gb</code>	Stable	Growing: memory leak; spiking: OOM risk
<code>throughput</code>	Consistent	Declining: thermal throttling or background load

Table 11: Per-step profiling metrics interpretation

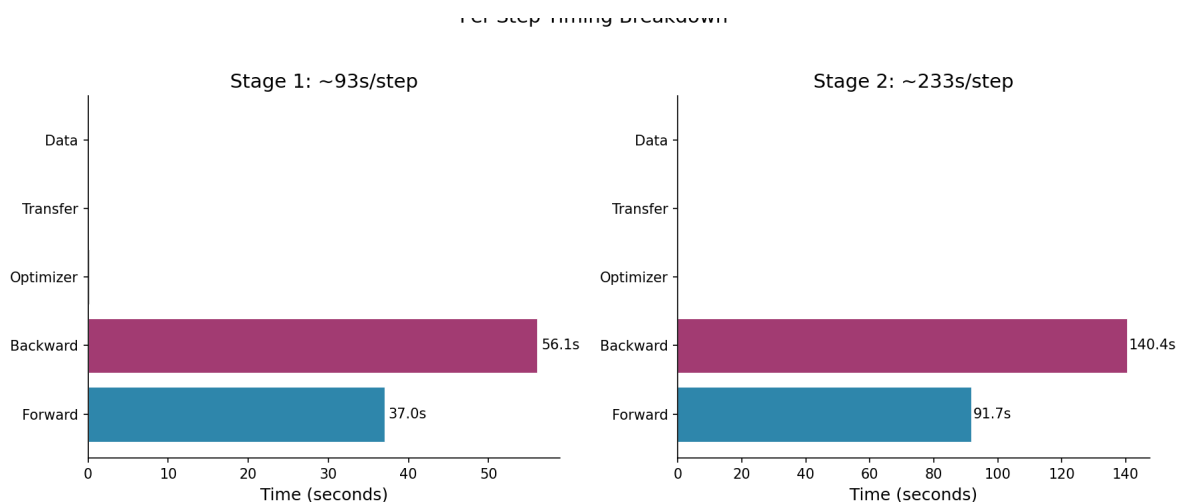


Figure 5: Per-step timing breakdown for Stage 1 vs Stage 2. The backward pass dominates in both stages ($\sim 60\%$ of step time). Stage 2 steps take $\sim 2.5\times$ longer due to longer sequences and more frames.

12.1.3 CSV Logging

Every metric is appended to a CSV file on every step:

```

1 step,loss,learning_rate,t_data,t_transfer,t_forward,t_backward,
2   t_optimizer,t_step,throughput,gpu_mem_gb,timestamp
3 1000,0.234,3.92e-05,0.003,0.048,37.5,58.1,0.135,95.8,0.334,
4 11.9,2025-12-10T14:23:01

```

Listing 15: Sample training CSV output

This data is essential for post-hoc analysis. When a crash happens at 3 AM, you can look at the CSV to see whether step times were increasing, memory was growing, or throughput was

declining before the crash.

12.2 Key Health Indicators

Metric	Healthy Range	Warning Signs
Training loss	Decreasing	Sudden spikes, NaN, plateau
Eval loss	Decreasing	Increasing (overfitting)
Perplexity	< 2.0	> 5.0 indicates problems
Learning rate	Following schedule	Stuck at 0 or not decaying
GPU memory	Stable	Growing or spiking
Step time	Consistent ($\pm 5\%$)	Sudden increases

Table 12: Training health indicators

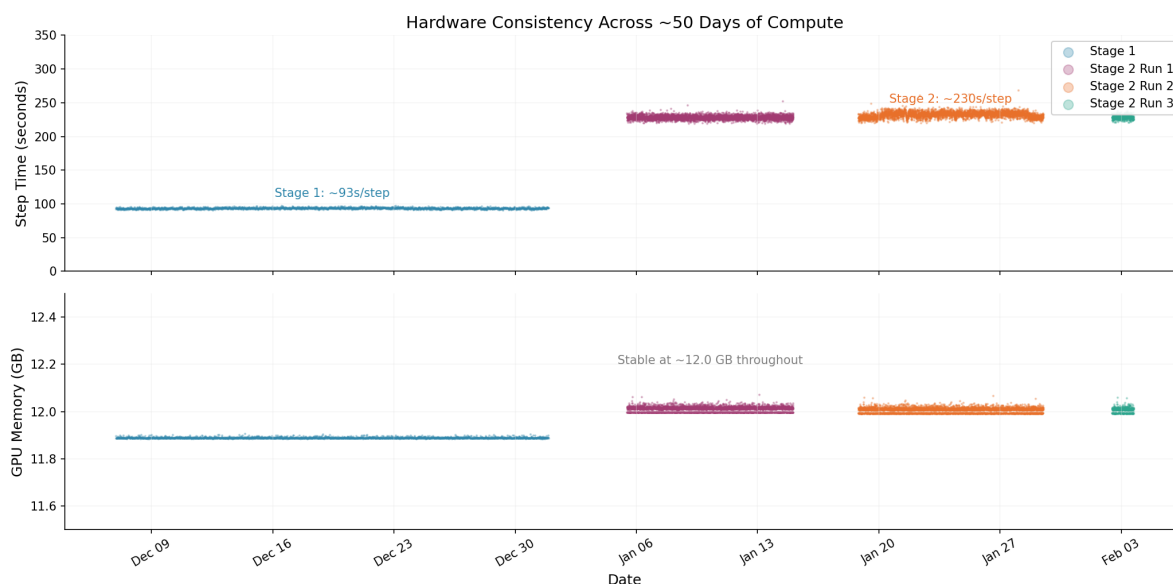


Figure 6: Step time and GPU memory over the entire training run. Both metrics remain remarkably stable throughout, confirming the crashes were caused by occasional memory spikes on extreme samples rather than gradual degradation.

12.3 Verify Checkpoint Loading

Warning

When resuming from a checkpoint, many training frameworks will **silently start from scratch** if the checkpoint path is invalid. Without explicit verification, you could waste hours of compute before noticing.

Always log checkpoint loading explicitly and sanity-check the first step:

```

1 def _load_checkpoint(self, path):
2     # Load adapter weights
3     adapter_state = load_file(path / "adapter_model.safetensors")
4     set_peft_model_state_dict(self.model, adapter_state)
5     logger.info(f"Loaded adapter weights from {path}")

```

```

6
7     # Load training state
8     state = torch.load(path / "training_state.pt")
9     self.global_step = state["global_step"]
10    logger.info(f"Resumed from checkpoint: step={self.global_step}")
11
12    # CRITICAL: Verify loss is consistent with pre-crash
13    # If step 1 loss is >> expected, checkpoint didn't load!

```

Listing 16: Checkpoint loading verification

The sanity check: If your model was at loss 0.01 before the crash and step 1 after restart shows loss 1.26, the checkpoint didn't load. Catching this in the first few minutes saves hours of wasted compute.

12.4 Automated Health Monitoring

On a cluster, the job scheduler monitors your training and restarts on failure. On a single machine, crashes can go undetected for hours—until you happen to check in.

Key Insight

A simple watchdog process that runs every 15–30 minutes can detect training failures within minutes and send notifications. This is trivial to set up and could save days over a multi-week run.

```

1 #!/bin/bash
2 # Check if training is still running
3 if ! pgrep -f "train.py" > /dev/null; then
4     LAST_STEP=$(tail -1 /path/to/train_metrics.csv)
5     curl -X POST "$SLACK_WEBHOOK" \
6         -d "{\"text\":\"\\nTraining stopped! Last: $LAST_STEP\\n\"}"
7 fi

```

Listing 17: Simple training watchdog (run via cron)

More sophisticated monitoring can track metric trends—alert if GPU memory is climbing, step time is increasing, or loss has gone to NaN—giving you early warning before a crash occurs.

Rule: If you're not monitoring it, you're not running it.

13 Debugging Crashes on New Hardware

13.1 Our Crash Pattern

During Stage 2 training, we experienced three system-level crashes that required hard reboots. The pattern was telling:

Crash	Stopped At	Steps Lost	Root Cause	How Found
1st	Step 27,186	2,186	GPU OOM	kern.log
2nd	Step 28,929	429	GPU OOM	Inferred

Table 13: Stage 2 crash history

Key observation: Stage 1 ran for 25 days with **zero crashes**. All crashes occurred during Stage 2. The difference? Stage 2 uses $2.5\times$ longer sequences and more frames, creating occasional memory spikes.

13.2 System Diagnostic Tools

13.2.1 dmesg: The First Check (But Often Useless)

After a crash requiring reboot, `dmesg` shows the *current* boot's messages, not the crash that caused the reboot:

```
1 $ dmesg | tail -20
2 # Shows current boot messages, NOT the crash
3 # nvidia-modeset errors here are display-related, NOT training OOM!
```

Listing 18: `dmesg` after reboot - only shows current boot

Warning

`nvidia-modeset`: ERROR: Failed to allocate KBPS Iso errors in `dmesg` are about **display output bandwidth**, not GPU compute memory. Do not confuse these with training OOMs—they are a red herring.

13.2.2 journalctl: Find Previous Boot Logs

The real diagnostic gold is in the previous boot's logs:

```
1 # List all boots (previous boots have negative indices)
2 $ journalctl --list-boots
3 -1 b7526ca7 Thu 2026-01-15 ... Thu 2026-01-29 # Previous
4 0 376d6ae2 Mon 2026-02-02 ... Mon 2026-02-02 # Current
5
6 # Check previous boot's kernel messages
7 $ journalctl -b -1 -k | grep -i "oom\|error\|fail\|nvrml"
```

Listing 19: Using `journalctl` to find crash evidence

13.2.3 Kernel Logs: The Smoking Gun

This is how we found our root cause. In `/var/log/kern.log.1` (the previous boot's kernel log):

```
1 2026-01-28T14:34:16 NVRM: Check failed: Out of memory
2 [NV_ERR_NO_MEMORY] @ mem_desc.c:1359
3 2026-01-28T14:35:06 fill_page_cache_func hogged CPU
4 for >20000us 4 times
5 2026-01-28T14:36:19 NVRM: Out of memory (3 more times)
6 2026-01-28T14:36:44 NVRM: Out of memory (2 more times)
```

Listing 20: The GPU OOM evidence from kernel logs

The story: The NVIDIA driver ran out of memory, the CPU started thrashing trying to handle the memory pressure (`fill_page_cache_func` hogged CPU), and the system eventually became unresponsive—requiring a hard reboot.

13.2.4 Thermal Monitoring

```
1 # Check thermal zones (values in millidegrees Celsius)
2 $ cat /sys/class/thermal/thermal_zone*/temp
3 79500 # 79.5C - warm but safe
4 69800 # 69.8C
5 70700 # 70.7C
6
7 # Continuous monitoring during training
8 $ watch -n 1 'cat /sys/class/thermal/thermal_zone*/temp'
```

Listing 21: Monitoring GPU thermals

For sustained GPU training, temperatures below 85°C are safe. Throttling typically starts at 85–90°C. In our case, thermals were **not** the issue—confirmed by monitoring showing 79.5°C peak.

13.3 The Root Cause: Unified Memory and Variable-Length Sequences

13.3.1 Why Stage 2 Crashed But Stage 1 Didn't

Parameter	Stage 1	Stage 2
max_frames	4	10
max_length	2048	4096
Eval batch size	8	4 (reduced from 8)
Step time	~93s	~230s

Table 14: Stage 1 vs Stage 2 memory pressure comparison

Stage 2's longer sequences occasionally spike memory beyond what the system can handle. On the DGX Spark's unified memory architecture (128GB shared between CPU and GPU), there is no separate swap space—when GPU memory fills up, the entire system hangs rather than producing a clean CUDA OOM error.

13.3.2 The Eval Batch Size Trap

Key Insight

The first crash occurred during evaluation, not training. Our eval batch size was 4 (2× training batch size). Combined with Stage 2's longer sequences, eval created larger memory spikes than training. This is easy to overlook because eval doesn't use gradients—but it processes more samples simultaneously.

Fix: Set eval batch size $\leq$ training batch size, especially for long-sequence stages.

13.3.3 Memory Headroom

We initially set `max_memory_gb: 115` on a 128GB system (90% utilization). This left only 13GB of headroom—not enough for worst-case samples. After identifying the OOM pattern, we reduced to `max_memory_gb: 100` (78% utilization), giving 28GB of headroom.

Rule of thumb for unified memory systems: Leave at least 20% headroom. Variable-length sequences create unpredictable memory spikes.

13.4 Debugging Tensor Mismatches and NaN Losses

During our initial Stage 2 launch, we encountered NaN losses caused by a tensor shape mismatch:

```
1 warning: The size of tensor a (4081) must match the size of
2   tensor b (8192), input_embeds[selected].shape=
3   torch.Size([4081, 3584]), vit_embeds.shape=
4   torch.Size([8192, 3584])
```

Listing 22: The NaN-causing tensor mismatch warning

Root cause: With `max_frames=16`, visual tokens alone consumed 4,096 tokens (16×256)—exactly equal to `max_length=4096`. After adding conversation text, sequences were truncated, cutting off image placeholder tokens. The vision encoder produced embeddings for all 16 frames, but the text contained slots for fewer—a mismatch that produced NaN gradients.

Fix: Reduced `max_frames` from 16 to 10, ensuring sufficient token budget for both visual and text content:

$$10 \text{ frames} \times 256 \text{ tokens/frame} = 2,560 \text{ visual tokens} + 1,536 \text{ for text} = 4,096 \quad (8)$$

Warning

Always verify: $(\text{max_frames} \times \text{tokens/frame}) + \text{max text length} \leq \text{max_length}$. Truncation of image placeholder tokens causes silent but catastrophic mismatches.

14 Evaluation and Validation

14.1 10% Subset Validation

Before full-scale training, we validated the pipeline on 10% of the data:

Stage	Eval Loss	Perplexity	Training Time
Stage 1	0.284	1.329	~2.4 days
Stage 2	0.034	1.034	~1.8 days

Table 15: 10% subset validation results

Observations:

- Consistent improvement throughout Stage 1 with no overfitting
- Stage 2 perplexity near 1.0 indicates high confidence on multi-turn reasoning
- Eval loss decreased monotonically, suggesting the model could benefit from more data

14.2 Full-Scale Results

14.2.1 Stage 1: Visual Grounding (100% Data)

Metric	10% Run	Full Run	Improvement
Eval Loss	0.284	0.125	56% better
Perplexity	1.329	1.133	—
Training Steps	2,259	22,593	—
Training Time	~2.4 days	~25 days	—

Table 16: Stage 1 full training results

Training dynamics:

- Eval loss decreased monotonically from 0.322 $\rightarrow$ 0.125 with no plateau
- Step time: ~93s (consistent throughout)
- GPU memory: 11.9 GB (stable)

- Model continued learning throughout without overfitting

Key Insight

The full dataset delivered substantial gains over the 10% validation run, confirming that the model benefits from more unique samples rather than multiple epochs over less data.

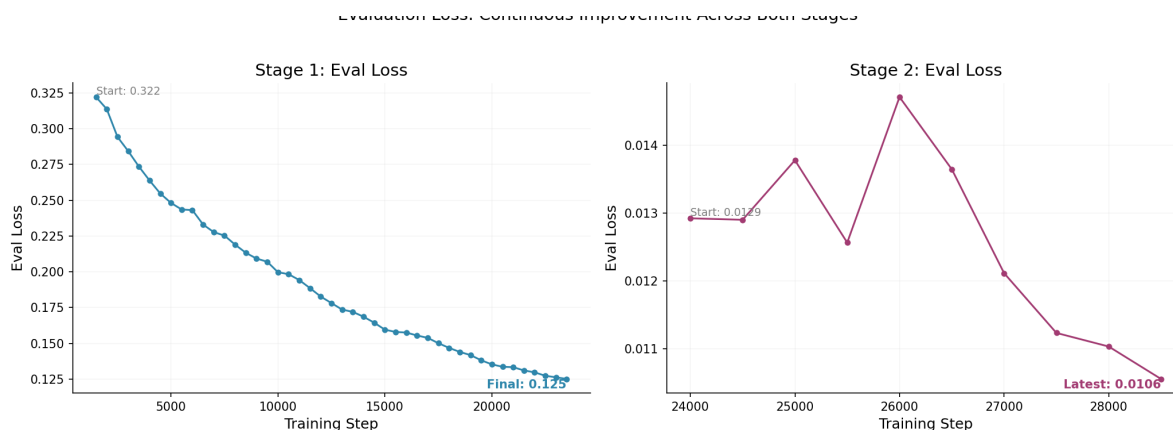


Figure 7: Evaluation loss over training steps. Stage 1 (left) shows steady improvement from 0.322 to 0.125 over 23K steps. Stage 2 (right) continues improving, with eval loss reaching 0.0106 and still decreasing.

15 Battle-Tested Pitfalls and How to Avoid Them

These are not hypothetical—every item below cost us hours or days of compute.

15.1 Data Preprocessing Pitfalls

1. **Wrong image size:** We initially preprocessed at 288×448 , requiring expensive BICUBIC resize on every batch. Pre-resizing to 448×448 during preprocessing and switching to BILINEAR interpolation for any remaining resizes dramatically improved data loading.
2. **Missing image tokens:** If `<image>` count doesn't match actual images, InternVL produces tensor mismatches. Some multi-turn Stage 2 samples had image markers only in the first turn—the collator must handle this.
3. **Token budget overflow:** With $16 \text{ frames} \times 256 \text{ tokens} = 4,096$ image tokens and `max_length=4096`, zero budget remained for text. Truncation silently removed image placeholders, causing NaN losses.
4. **Data leakage:** Keep train/val splits consistent across stages. Both stages should split on the same video IDs.

15.2 Checkpoint and Recovery Pitfalls

1. **save_steps too large:** 5,000 steps at 227s/step = 13 days between checkpoints. One crash cost us 6 days.
2. **No final checkpoint:** Training at 2,259 steps with `save_steps: 1000` means the last save was at step 2,000—missing 259 steps. Always save on completion.

3. **Silent restart on bad checkpoint path:** Many frameworks silently start from scratch if the path is invalid. Always verify by checking the first step’s loss matches expectations.
4. **Not saving optimizer/scheduler state:** Without optimizer state, resumed training uses a fresh optimizer, causing training instability. Without scheduler state, learning rate resets to warmup.
5. **No failure monitoring:** Without a watchdog, crashes go undetected for hours. A simple cron job checking process liveness can alert you within minutes.

15.3 Memory and Stability Pitfalls

1. **Eval batch size larger than train:** We used `eval_batch_size=4` with `train_batch_size=2`. Eval doesn’t store gradients but processes $2\times$ more samples simultaneously—this caused our first OOM crash.
2. **Insufficient memory headroom:** 90% utilization (115/128GB) left no room for worst-case samples. 78% (100/128GB) proved stable.
3. **Unified memory hangs instead of OOMing:** On DGX Spark, GPU OOM causes system freeze, not a Python exception. The only evidence is in kernel logs from the previous boot.
4. **Assuming Flash Attention works:** We spent days compiling FA2 for Blackwell (sm_121). It ran 2% *slower* than SDPA. Always benchmark.

15.4 Stage Transition Pitfalls

1. **Not resetting the LR scheduler:** When transitioning from Stage 1 to Stage 2, load weights but use a fresh cosine schedule. Otherwise, the learning rate continues decaying from Stage 1’s end—near zero.
2. **Not resetting best_eval_loss:** Stage 2’s loss scale differs from Stage 1. Keep the old best loss and you’ll never save a “best” checkpoint.
3. **Wrong “best” checkpoint:** “Best” is based on eval loss, which may not correspond to the last checkpoint. Verify what “best” actually points to.

16 Results and Lessons Learned

16.1 Final Results Summary

Metric	Stage 1 Final	Stage 2 Final
Eval Loss	0.125	(In progress)
Perplexity	1.133	(In progress)
Training Time	25 days	~22 days

Table 17: Final training results

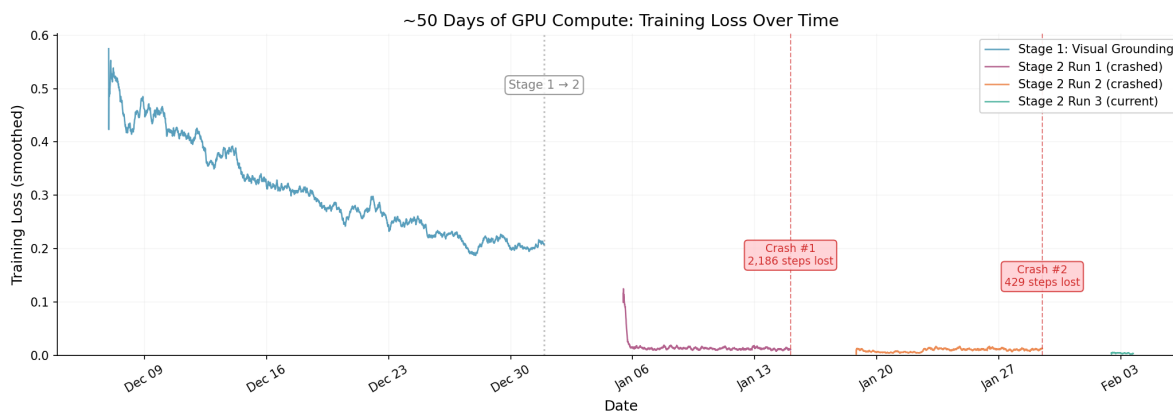


Figure 8: The complete training journey by calendar date, spanning Dec 6, 2025 to Feb 3, 2026. Stage 1 shows steady loss descent over 25 days. Stage 2 runs at much lower loss but suffered two OOM crashes (red dashed lines), requiring restarts from the last checkpoint.



Figure 9: Training loss by step across both stages. The stage transition at step 23,594 shows the dramatic shift from visual grounding (loss ~ 0.13) to multi-turn reasoning (loss ~ 0.02). Overlapping Stage 2 runs from crash recovery are visible as repeated segments.

16.2 Key Lessons Learned

1. **Curriculum learning works:** 48% improvement on visual grounding, 26% on reasoning
2. **More data beats more epochs:** Full dataset showed continued improvement without overfitting
3. **Benchmark everything:** Flash Attention wasn't faster on our hardware
4. **QLoRA enables accessibility:** 8B model training in ~ 12 GB VRAM
5. **Preprocessing is critical:** Garbage in, garbage out
6. **Checkpoint frequency is a survival strategy:** Convert `save_steps` to wall-clock hours. If > 24 h, increase frequency.
7. **Design for crashes, not success:** Two OOM crashes during Stage 2, zero during Stage 1.
 1. The difference was memory headroom.

8. **Log everything to CSV:** When a crash happens at 3 AM, post-hoc analysis of per-step metrics is your only diagnostic tool.
9. **Verify checkpoint loading explicitly:** Silent restarts waste hours of compute. Log adapter loading and sanity-check first-step loss.
10. **System logs outlive Python logs:** `journalctl` and `/var/log/kern.log.1` contain the real crash evidence after a reboot.
11. **Monitor your training:** A simple watchdog cron job can detect failures in minutes instead of hours. If you're not monitoring it, you're not running it.

17 Appendix: Running the Code

17.1 Installation

```

1 # Clone repository
2 git clone https://github.com/giricme/vlm-ft.git
3 cd vlm-ft
4
5 # Install dependencies
6 pip install -e .
7
8 # Download data
9 python -m vlmft.data.download_robovqa --output-dir data/robovqa/raw

```

Listing 23: Installation commands

17.2 Preprocessing

```

1 # Preprocess RoboVQA
2 python -m vlmft.data.preprocess_robovqa \
3     --data-dir data/robovqa/raw \
4     --output-dir data/robovqa/processed \
5     --workers 8
6
7 # Verify preprocessing
8 python -m vlmft.data.preprocess_robovqa \
9     --verify \
10    --output-dir data/robovqa/processed

```

Listing 24: Preprocessing commands

17.3 Training

```

1 # Stage 1: Visual Grounding
2 python -m vlmft.scripts.train --config configs/stage1_full.yaml
3
4 # Stage 2: Multi-Turn Reasoning (after Stage 1 completes)
5 python -m vlmft.scripts.train --config configs/stage2_full.yaml
6
7 # Evaluation
8 python -m vlmft.scripts.eval \
9     --checkpoint experiments/stage1/checkpoints/final \
10    --config configs/stage1.yaml

```

Listing 25: Training commands

17.4 Configuration Override

```
1 # Override specific parameters
2 python -m vlmft.scripts.train \
3     --config configs/stage1.yaml \
4     data.subset_ratio=0.1 \
5     batch_size=4 \
6     optimizer.learning_rate=2e-5
```

Listing 26: Configuration override example

18 Conclusion

Fine-tuning Vision-Language Models for robotics is now accessible thanks to QLoRA and curriculum learning. But the gap between “training works on a small test” and “training completes successfully over weeks” is enormous. The key insights from this project:

1. **Start with foundations:** Visual grounding before complex reasoning
2. **Use QLoRA:** Makes large model training feasible
3. **Preprocess carefully:** Data quality determines success
4. **Validate incrementally:** 10% runs before full training
5. **Engineer for failure:** Aggressive checkpointing, comprehensive logging, explicit verification
6. **Know your system tools:** `dmesg`, `journalctl`, kernel logs, and thermal monitoring are as important as your training script

The trained model demonstrates strong performance on robotics VQA tasks, with significant improvements from curriculum learning. But equally valuable are the operational lessons: the checkpoint strategy that saved days of compute, the logging that enabled root-cause analysis, and the system debugging that identified memory pressure as the failure mode. These lessons generalize to any long-running training workload.

Code: <https://github.com/giricme/vlm-ft>
Model weights freely available via HuggingFace (links in repo)